Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Баранова Д.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата:06.11.26

Москва, 2026

Постановка задачи

Вариант 2.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- ssize_t read(int fd, void *buf, size_t count) — чтение данных из файлового дескриптора.
- ssize_t write(int fd, const void *buf, size_t count) — запись данных в файловый дескриптор.
- int shm_open(const char *name, int oflag, mode_t mode); — открыть/создать объект разделяемой памяти.
- int ftruncate(int fd, off_t length); — задаёт размер объекта (у созданного shm он 0).
- void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset); — отображает объект в адресное пространство процесса
- int munmap(void *addr, size_t length); — снимает отображение.
- int shm_unlink(const char *name); — удаляет имя объекта из пространства имён.
- sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value); — открыть/создать именованный семафор.
- int sem_wait(sem_t *sem); — декрементирует счетчик, если счётчик 0 — блокируется, пока кто-то не сделает sem_post.
- int sem_post(sem_t *sem); — инкрементирует счётчик.
- int sem_close(sem_t *sem); — закрывает семафор в текущем процессе
- int sem_unlink(const char *name); — удаляет имя семафора.
- int open(const char *pathname, int flags, mode_t mode) — открытие/создание файла. Флаги O_WRONLY | O_CREAT | O_TRUNC обеспечивают запись, создание при отсутствии и очистку при существовании.
- int close(int fd) — закрытие файлового дескриптора.
- int execv(const char *file, const char *arg, ...) — замена образа текущего процесса новой программой.
- pid_t waitpid(pid_t pid, int *wstatus, int options); — ожидает завершения дочернего процесса.

Алгоритм parent.cpp:

Считать имя файла с терминала через read.
Проверить ввод: -1 — ошибка, 0 — пусто. При ошибке — выход.
Считать имя SHM-объекта, проверить ввод.
Открыть SHM-объект и сделать отображение в адресное пространство процесса.
Открыть два именованных семафора: empty – если счетчик больше 0, то в общую память можно писать, в ней лио нет данных, либо они уже обработаны; full – если счетчик больше 0, в общей памяти есть данные для обработки дочерним процессом.
Создать дочерний процесс fork().
В цикле считывать строки с числами из stdin и писать в общую память. По EOF выйти из цикла.
Дождаться завершения ребенка.
Удалить имена и закрыть семафоры и общую память.

Алгоритм child.cpp:

Открыть SHM и семафоры.
Открыть файл.
В цикле читать строки из SHM.
Каждую строку разбить на токены через strtok, каждый токен преобразовать в double через strtod, сложить.
Сумму преобразовать в строку через std::to_string, добавить \n, записать в файл через write.
По EOF закрыть файл, SHM и семафоры, снять отображение памяти в адресное пространство.

Код программы

parent.cpp

```
#include <unistd.h>
#include <sys/wait.h>
#include <cstdlib>
#include <fcntl.h>
#include <sys/types.h>
#include <cstring>
#include <sys/mman.h>
#include <semaphore.h>
#include <cstdio>
#include <cstdint>

#define MAX_NAME 4096
#define SHM_SIZE 4096

ssize_t read_line(char* buf, size_t max){
        size_t i = 0;
```

```
        char c;
        while( i < max - 1){
                ssize_t n = read(STDIN_FILENO, &c, 1);
                if (n < 0) return -1;
                if (n == 0) break;
                buf[i++] = c;
                if (c == '\n') break;
        }
        buf[i] = '\0';
        return i;
}

int main(){
        char filename[MAX_NAME];
        write(STDOUT_FILENO, "Enter filename: ", 16);
        ssize_t n = read(STDIN_FILENO, filename, MAX_NAME-1);
        if (n == -1){
                write(STDERR_FILENO, "read error\n", 11);
                std::exit(EXIT_FAILURE);
        }
        if (n == 0){
                write(STDERR_FILENO, "no filename\n", 12);
                std::exit(EXIT_FAILURE);
        }
        if (filename[n - 1] == '\n'){
               filename[n - 1] = '\0';
               n--;
        }else filename[n] = '\0';
        if (n == 0){
                 write(STDERR_FILENO, "empty filename\n", 15);
                 std::exit(EXIT_FAILURE);
        }

        char shm_name[MAX_NAME - 6];
        write(STDOUT_FILENO, "Enter shared memory object name: ", 33);
        n = read(STDIN_FILENO, shm_name, sizeof(shm_name)-1);
        if (n == -1){
                write(STDERR_FILENO, "read error\n", 11);
                std::exit(EXIT_FAILURE);
        }
        if (n == 0){
                write(STDERR_FILENO, "no name\n", 8);
                std::exit(EXIT_FAILURE);
        }
        if (shm_name[n - 1] == '\n'){
               shm_name[n - 1] = '\0';
               n--;
        }else shm_name[n] = '\0';
        if (n == 0){
                write(STDERR_FILENO, "empty name\n", 11);
                std::exit(EXIT_FAILURE);
        }
        char shm_full[sizeof(shm_name) + 1];
        std::snprintf(shm_full, sizeof(shm_full), "/%s", shm_name);
```

```
        int shm = shm_open(shm_full, O_RDWR | O_CREAT | O_TRUNC, 0600);
        if (shm == -1){
                write(STDERR_FILENO, "failed to open SHM\n", 19);
                std::exit(EXIT_FAILURE);
        }
        if (ftruncate(shm, SHM_SIZE) == -1){
                write(STDERR_FILENO, "failed to resize SHM\n", 21);
                std::exit(EXIT_FAILURE);
        }

        char* shm_buf = (char*)mmap(NULL, SHM_SIZE, PROT_WRITE, MAP_SHARED, shm, 0);
        if (shm_buf == MAP_FAILED) {
                write(STDERR_FILENO, "failed to map SHM\n", 18);
                std::exit(EXIT_FAILURE);
        }

        char empty_name[MAX_NAME];
        std::snprintf(empty_name, sizeof(empty_name), "%s-sem1", shm_full);
        sem_t* empty = sem_open(empty_name, O_CREAT, 0600, 1);
        if (empty == SEM_FAILED){
                write(STDERR_FILENO, "failed to create semaphore\n", 27);
                std::exit(EXIT_FAILURE);
        }

        char full_name[MAX_NAME];
        std::snprintf(full_name, sizeof(full_name), "%s-sem2", shm_full);
        sem_t* full = sem_open(full_name, O_CREAT, 0600, 0);
        if (full == SEM_FAILED){
                write(STDERR_FILENO, "failed to create semaphore\n", 27);
                std::exit(EXIT_FAILURE);
        }

        pid_t pid = fork();
        if (pid == -1){
                write(STDERR_FILENO, "fork error\n", 11);
                std::exit(EXIT_FAILURE);
        }else if (pid == 0){
                char* argv[] = {"./child", filename, shm_full, empty_name, full_name,
NULL};
                execv("./child", argv);
                write(STDERR_FILENO, "exec error\n", 11);
                std::exit(EXIT_FAILURE);
        }

        bool running = true;
        write(STDOUT_FILENO, "Enter numbers: ", 15);
        while (running) {
                char buf[SHM_SIZE - sizeof(uint32_t)];
                ssize_t bytes = read_line(buf, sizeof(buf));

                if (bytes == -1) {
                        write(STDERR_FILENO, "failed to read from standard input\n", 35);
                        std::exit(EXIT_FAILURE);
```

```
                }

                sem_wait(empty);
                uint32_t* length = (uint32_t*)shm_buf;
                if (bytes > 0) {
                        char* text = shm_buf + sizeof(uint32_t);
                        *length = bytes;
                        memcpy(text, buf, bytes);
                } else {
                        *length = UINT32_MAX;
                        running = false;
                }
                sem_post(full);
        }

        waitpid(pid, NULL, 0);
        sem_unlink(empty_name);
        sem_close(empty);
        sem_unlink(full_name);
        sem_close(full);
        munmap(shm_buf, SHM_SIZE);
        shm_unlink(shm_full);
        close(shm);
}
```

child.cpp

```
#include <unistd.h>
#include <cstdlib>
#include <cstring>
#include <fcntl.h>
#include <string>
#include <cstdint>
#include <fcntl.h>
#include <sys/mman.h>
#include <semaphore.h>

#define SHM_SIZE 4096

int main(int argc, char *argv[]){
    if (argc != 5){
        write(STDERR_FILENO, "wrong amount of arguments\n", 26);
        std::exit(EXIT_FAILURE);
    }
    char* filename = argv[1];
    char* shm_name = argv[2];
    char* empty_name = argv[3];
    char* full_name = argv[4];

    int shm = shm_open(shm_name, O_RDWR, 0);
    if (shm == -1){
        write(STDERR_FILENO, "failed to open SHM\n", 19);
        std::exit(EXIT_FAILURE);
    }
```

```
    char* shm_buf = (char*)mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm,
0);
    if (shm_buf == MAP_FAILED) {
        write(STDERR_FILENO, "failed to map SHM\n", 18);
        std::exit(EXIT_FAILURE);
    }

    sem_t* empty = sem_open(empty_name, 0);
    if (empty == SEM_FAILED){
        write(STDERR_FILENO, "failed to open semaphore\n", 25);
        std::exit(EXIT_FAILURE);
    }

    sem_t* full = sem_open(full_name, 0);
    if (full == SEM_FAILED){
        write(STDERR_FILENO, "failed to open semaphore\n", 25);
        std::exit(EXIT_FAILURE);
    }

    int fd = open(filename, O_CREAT | O_RDWR | O_TRUNC, 0600);
    if(fd == -1){
        write(STDERR_FILENO, "failed to open file descriptor\n", 31);
        std::exit(EXIT_FAILURE);
    }

    bool running = true;
    while(running){

        sem_wait(full);

        uint32_t* length = (uint32_t*)shm_buf;
        char* text = shm_buf + sizeof(uint32_t);

        if (*length == UINT32_MAX){
            running = false;
        }else if (*length > 0){
            char buffer[SHM_SIZE - sizeof(uint32_t) + 1];
            memcpy(buffer, text, *length);
            buffer[*length] = '\0';
            *length = 0;

            char* token = strtok(buffer, " \t");
            double sum = 0;
            while(token != NULL){
                char *end;
                double value = strtod(token, &end);
                if (end == token){
                    write(STDERR_FILENO, "bad number: ", 12);
                    write(STDERR_FILENO, token, strlen(token));
                    write(STDERR_FILENO, ", skipped\n", 10);
                }else{
                    sum += value;
                }
```

```
                token = strtok(NULL, " \t");
            }
        std::string s =  std::to_string(sum)+"\n";
        write(fd, s.c_str(), s.size());
        }
        sem_post(empty);
    }

    close(fd);
    sem_close(empty);
    sem_close(full);
    munmap(shm_buf, SHM_SIZE);
    close(shm);
}
```

Протокол работы программы

```
 diana@DESKTOP-NSHR21B:~/lab1$ ./newparent
Enter filename: res
Enter shared memory object name: d
Enter numbers: 986523.645
8465.56
215648.0 4685312.1653 0.32      8465.21
531 4.2
diana@DESKTOP-NSHR21B:~/lab1$ cat res
986523.645000
8465.560000
4909425.695300
535.200000
diana@DESKTOP-NSHR21B:~/lab1$ ./newparent
Enter filename: res
Enter shared memory object name: f
Enter numbers: diana@DESKTOP-NSHR21B:~/lab1$ cat res
diana@DESKTOP-NSHR21B:~/lab1$ ./newparent
Enter filename: res
Enter shared memory object name:
empty name
diana@DESKTOP-NSHR21B:~/lab1$ ./newparent
```

```
 Enter filename: res

Enter shared memory object name: g

Enter numbers: ervdf hbj8iuo09

bad number: ervdf, skipped

bad number: hbj8iuo09

, skipped

diana@DESKTOP-NSHR21B:~/lab1$ cat res

0.000000
```

Вывод

В ходе работы я научилась работать с разделяемой памятью и обеспечивать обмен данных между процессами с помощью file mapping. Реализована программа, в которой родительский процесс построчно передаёт числа через общий буфер, а дочерний вычисляет сумму чисел в каждой строке и записывает результат в файл, при этом два семафора empty/full обеспечивают корректную очерёдность доступа и исключают перезапись непрочитанных данных.